

Curso de Automatización con Java, Selenium, JUnit y Gherkin

Clase 4 – Interacciones básicas I

COLEGIO
UNIVERSITARIO



CCC
Centro de Capacitación
Continua

Temario

- ◆ Localizadores: id, name, css, xpath
- ◆ Acciones: click, sendKeys, getText
- ◆ Esperas implícitas



Práctica: automatización de formulario simple



¿Qué son los localizadores?

Los **localizadores (locators)** son las “direcciones” que utiliza Selenium para encontrar elementos en una página web (inputs, botones, textos, enlaces, etc.).

Tipo de localizador	Método Selenium	Ejemplo	Uso recomendado
id	<code>By.id("email")</code>	<code><input id="email"></code>	Rápido y único
name	<code>By.name("username")</code>	<code><input name="username"></code>	Común en formularios
className	<code>By.className("btn")</code>	<code><button class="btn"></code>	Para estilos comunes
tagName	<code>By.tagName("input")</code>	<code><input></code>	Menos específico
linkText	<code>By.linkText("Iniciar sesión")</code>	<code><a>Iniciar sesión</code>	Links de texto
partialLinkText	<code>By.partialLinkText("Iniciar")</code>	<code><a>Iniciar sesión</code>	Links parciales
cssSelector	<code>By.cssSelector("#login input[type='text']")</code>	-	Muy potente
xpath	<code>By.xpath("//input[@id='email']")</code>	-	Más flexible (pero menos legible)

CSS Selector (Cascading Style Selector)

Concepto

Los CSS *Selectors* provienen del mundo del diseño web (Hojas de Estilo en Cascada) y se usan para aplicar estilos a elementos. Selenium los aprovecha para identificar elementos de forma precisa, rápida y legible.

✓ Ventajas

- Más rápidos que XPath en la mayoría de los navegadores.
- Más cortos y legibles.
- Soportan combinaciones poderosas (`id`, `class`, `atributos`, jerarquías).

⚠ Desventajas

- No permiten navegar *hacia atrás* (de hijo a padre).
- No tienen tantas funciones de texto o posición como XPath.

Objetivo	Sintaxis CSS	Ejemplo
Por id	#idElemento	#login-button
Por clase	.nombreClase	.btn-primary
Por atributo	[atributo='valor']	[name='email']
Por atributo que contiene texto	[atributo*='parte']	[id*='user']
Por atributo que empieza con	[atributo^='prefijo']	[name^='pass']
Por atributo que termina con	[atributo\$='sufijo']	[id\$='name']
Por jerarquía descendente	div input	#form-login input
Por elemento hijo directo	div > input	.container > button
Combinación de clase y atributo	.btn[disabled='true']	—

```
WebElement campoUsuario = driver.findElement(By.cssSelector("input#user-name"));
WebElement botonLogin = driver.findElement(By.cssSelector("input[type='submit']"));
WebElement linkRecuperar = driver.findElement(By.cssSelector("a[href*='recover']"));
```

💡 *Consejo:* Si el HTML tiene atributos `data-*` personalizados (por ejemplo, `data-test='login-button'`), usalos — son ideales para automatización:

```
By.cssSelector("[data-test='login-button']");
```

XPath (XML Path Language)

Concepto

XPath proviene del mundo XML.

Permite navegar por toda la jerarquía del documento HTML (hacia arriba, abajo, o lateralmente) con gran precisión.

✓ Ventajas

- Extremadamente flexible.
- Permite moverse entre **padres, hijos, hermanos**.
- Soporta condiciones lógicas y funciones (`contains()`, `text()`, `starts-with()`).

⚠ Desventajas

- Más lentos que CSS (especialmente en Chrome).
- Más propensos a romperse si cambia el DOM.
- Menos legibles cuando son largos.

Objetivo	Sintaxis XPath	Ejemplo
Por atributo	<code>//tag[@atributo='valor']</code>	<code>//input[@id='email']</code>
Por múltiples condiciones	<code>//tag[@id='x' and @name='y']</code>	<code>//input[@id='email' and @type='text']</code>
Por texto exacto	<code>//tag[text()='Texto']</code>	<code>//button[text()='Enviar']</code>
Por texto parcial	<code>//tag[contains(text(),'Enviar')]</code>	<code>//a[contains(text(),'Olvidé')]</code>
Por atributo parcial	<code>//tag[contains(@id,'user')]</code>	<code>//input[contains(@id,'user')]</code>
Por descendencia	<code>//div/input</code>	<code>//form//button</code>
Por índice	<code>(//input[@type='text'])[2]</code>	—
Por hermano	<code>//input[@id='user']/following-sibling::label</code>	—
Por padre	<code>//label[@for='user']/parent::div</code>	—

```
WebElement usuario = driver.findElement(By.xpath("//input[@id='user-name']"));
WebElement boton = driver.findElement(By.xpath("//input[contains(@class,'btn_action')]"));
WebElement link = driver.findElement(By.xpath("//a[text()='Forgot Password?']"));
```

💡 *Truco útil:* cuando los elementos no tienen `id` ni `name`, podés usar combinaciones:

```
By.xpath("//div[@class='login-box']//input[@type='password']");
```

Comparativa CSS vs XPath

Característica	CSS Selector	XPath
Velocidad	Más rápido	Más lento
Legibilidad	Más simple	Más complejo
Soporta texto visible	✗ No	✓ Sí (text())
Permite navegar al padre	✗ No	✓ Sí
Usado por diseñadores front-end	✓	✗
Ideal para automatización estable	✓ Sí	✓ Sí, si se usa con cuidado

Acciones básicas con Selenium

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class InteraccionesBasicas {
    public static void main(String[] args) {
        System.setProperty("webdriver.chrome.driver", "C:\\\\drivers\\\\chromedriver.exe");
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();

        driver.get("https://www.saucedemo.com/");

        // Localizar elementos
        WebElement usuario = driver.findElement(By.id("user-name"));
        WebElement password = driver.findElement(By.id("password"));
        WebElement botonLogin = driver.findElement(By.id("login-button"));

        // Interacciones
        usuario.sendKeys("standard_user");
        password.sendKeys("secret_sauce");
        botonLogin.click();

        System.out.println("Título actual: " + driver.getTitle());

        driver.quit();
    }
}
```

Obtener información de los elementos

```
WebElement elemento = driver.findElement(By.id("user-name"));
System.out.println("Tag: " + elemento.getTagName());
System.out.println("Placeholder: " + elemento.getAttribute("placeholder"));
System.out.println("Visible: " + elemento.isDisplayed());
```

Acción	Método WebDriver / WebElement	Descripción	Ejemplo en Java
Abrir una página	<code>driver.get("URL")</code>	Abre la URL especificada en el navegador.	<code>driver.get("https://www.google.com");</code>
Obtener el título	<code>driver.getTitle()</code>	Devuelve el título de la página actual.	<code>System.out.println(driver.getTitle());</code>
Obtener URL actual	<code>driver.getCurrentUrl()</code>	Devuelve la dirección actual del navegador.	<code>System.out.println(driver.getCurrentUrl());</code>
Localizar elemento	<code>driver.findElement(By.id("..."))</code>	Busca un elemento usando un localizador.	<code>WebElement campo = driver.findElement(By.id("user-name"));</code>

Acción	Método WebDriver / WebElement	Descripción	Ejemplo en Java
Escribir texto	<code>sendKeys("texto")</code>	Escribe texto dentro de un input o textarea.	<code>campo.sendKeys("usuario123");</code>
Borrar texto	<code>clear()</code>	Limpia el contenido de un campo de texto.	<code>campo.clear();</code>
Click	<code>click()</code>	Hace click en un botón, enlace o checkbox.	<code>boton.click();</code>
Obtener texto	<code>getText()</code>	Devuelve el texto visible del elemento.	<code>String texto = label.getText();</code>

Acción	Método WebDriver / WebElement	Descripción	Ejemplo en Java
Verificar visibilidad	<code>isDisplayed()</code>	Devuelve true si el elemento está visible.	<code>if(boton.isDisplayed()) boton.click();</code>
Verificar si está habilitado	<code>isEnabled()</code>	Indica si el elemento está disponible para interacción.	<code>boton.isEnabled();</code>
Verificar si está seleccionado	<code>isSelected()</code>	Indica si un checkbox o radio está marcado.	<code>check.isSelected();</code>

Acción	Método WebDriver / WebElement	Descripción	Ejemplo en Java
Esperar (implícita)	<code>driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(x))</code>	Espera que los elementos aparezcan antes de lanzar error.	<code>driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));</code>
Maximizar ventana	<code>driver.manage().window().maximize()</code>	Amplía el navegador a pantalla completa.	<code>driver.manage().window().maximize();</code>
Cerrar ventana actual	<code>driver.close()</code>	Cierra la pestaña activa.	<code>driver.close();</code>
Cerrar todo el navegador	<code>driver.quit()</code>	Finaliza la sesión de WebDriver (todas las ventanas).	<code>driver.quit();</code>

Esperas implícitas

Problema común: a veces Selenium intenta interactuar antes de que el elemento esté disponible.

Solución: usar *esperas*.

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

Esta espera se aplica globalmente: Selenium esperará hasta 10 segundos a que los elementos aparezcan antes de lanzar una excepción.

Práctica

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua

Ejercicio guiado: formulario simple

Objetivo: automatizar el siguiente flujo:

1. Ingresar a la página <https://www.saucedemo.com/>
2. Completar usuario y contraseña.
3. Hacer click en “Ingresar”.
4. Imprimir el título o mensaje resultante.

Ejercicio adicional: automatizar un formulario de contacto

Usar una web demo como:

👉 <https://www.selenium.dev/selenium/web/web-form.html>

Flujo:

1. Completar nombre, email, mensaje.
2. Seleccionar opciones o marcar checkboxes.
3. Enviar formulario.
4. Validar mensaje de éxito.

Cierre

**COLEGIO
UNIVERSITARIO**



CCC

Centro de Capacitación
Continua

Buenas prácticas iniciales

- ✓ Usar `id` o `name` siempre que sea posible (más estables).
- ✓ Evitar `xpath` o `css` muy largos o dependientes de estilos.
- ✓ Limpiar entradas antes de escribir (`clear()` si es necesario).
- ✓ Maximizar la ventana (`driver.manage().window().maximize()`).

¿Preguntas?

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua

¡Gracias!

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua